2026 8th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA) | 979-8-3315-8150-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICHORA69329.2026.11537111

Pre-authorized and asynchronous Payment in Urban Taxi Mobility Apps: Reducing End-of-Trip Friction with an Event-Driven Microservices Architecture and Impact Analysis Using Production Data

Mustafa Ünlü
Research & Development
Bitaksi Mobil Teknoloji A.Ş.
Istanbul, Türkiye
mustafa.unlu@bitaksi.com

Engin Tanrıkulu
Research & Development
Bitaksi Mobil Teknoloji A.Ş.
Istanbul, Türkiye
engin.tanrikulu@bitaksi.com

Akın Kırcı
Research & Development
Bitaksi Mobil Teknoloji A.Ş.
Istanbul, Türkiye
akin.kirci@bitaksi.com

Emir Esmerdağ
Research & Development
Bitaksi Mobil Teknoloji A.Ş.
Istanbul, Türkiye
emir.esmerdag@bitaksi.com

Emre Gerçek
Research & Development
Bitaksi Mobil Teknoloji A.Ş.
Istanbul, Türkiye
emre.gercek@bitaksi.com

Tuna Çakar
Computer Engineering
MEF University
Istanbul, Türkiye
cakart@mef.edu.tr

Abstract*—This study presents an "pre-authorized and asynchronous payment" approach developed to reduce the waiting time, uncertainty, and operational congestion caused by having to wait for payment confirmation at the end of a trip in urban taxi mobility applications. In the existing synchronous flow, delays caused by third-party payment services (including timeouts), network instability, and insufficient card limit/balance both slow down the passenger's exit from the vehicle and delay the driver's transition to the next trip. In the proposed solution, trip closure is decoupled from the payment process: the TripEnded event is published via Kafka, the Payment Service performs collection in the background, and the client flow is not blocked. On the authorization side, a Redis-based TTL cache reduces unnecessary repeated authorizations; and a settlement decision logic manages differences between the authorized amount and the final charged amount. In risky cases, the flow is redirected to 3D Secure; for failed collections, staged backup-card attempts, debt recording, and scheduled automatic collection (retry) are applied. In the evaluation using production data, the average client blocking time decreased from 46.33 seconds to 2.31 seconds, and P95/P99 values improved significantly. In field measurements, the vehicle exit (drop-off) time decreased from 4.67 seconds to 1.43 seconds; and the collection rate within the first hour increased from 38.56% to 79.88%. On the operations side, the number of calls decreased by 20.24% as critical complaints dropped. The results show that an asynchronous "non-blocking collection" design strengthens both user experience and collection continuity. During gradual rollout, the pre-authorized payment infrastructure was used in about 25% of card trips, and as this rate increased, friction at the moment of trip closure systematically decreased. Since reducing approval steps can increase malicious-use risk, device/GSM-based limits, anomaly detection, and fraud-flagging rules were introduced, and stricter verification policies were applied for suspicious transactions. The architecture was supported with observability (logs/metrics/alerts) and CI/CD processes to achieve a scalable trip-closure payment model.

Keywords— Pre-authorized payment, Asynchronous collection, Event-driven architecture, Microservices, Kafka, Redis cache, 3D Secure.

I. INTRODUCTION

One of the moments when users experience the most "friction" in urban transportation apps is when the payment step becomes slow and uncertain right after the trip ends. Especially in taxis, waiting for a physical POS device, scanning a QR code, or giving manual approval on a mobile screen both delays the passenger's exit and slows down the driver's move to the next trip. Based on this need, an R&D-focused effort was carried out. The main goal of the study is to remove extra interactions required for payment at the end of the trip and make the closure moment as fast and smooth as possible.

In the current synchronous payment approach, because trip completion and payment confirmation progress at the same time, both the passenger and driver applications can remain in a "waiting" state until the process finishes. Field data and feedback show that delays originating from third-party payment services (including timeouts) can extend from seconds to minutes; in cases of insufficient card limit/balance, the process restarts and can turn into a tense in-car cycle of searching for an alternative payment method; and in-vehicle network instability can lead to repeated attempts and refund processes. These problems are not only technical delays; they also create direct business impacts such as reduced trust in digital payment, lower driver productivity, and failing to meet the "smooth closure" expectation that modern apps aim to deliver.

979-8-3315-8150-3/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

This paper discusses the "pre-authorized payment" approach developed to solve these problems and the technical design behind it. In the proposed structure, automatic authorization and collection operations are performed in the background at the start and end of the trip using the payment method registered in the app, so the passenger can exit without waiting. Steps such as payment confirmation, invoice generation, and driver earnings calculation are completed asynchronously and reflected on the driver's screen. This aims to improve user experience and also speed up operational flow in high-traffic areas (e.g., airports, shopping malls).

Technically, the solution is built on an event-driven microservices architecture due to high availability and consistency requirements, and it is supported by asynchronous messaging and caching strategies to reduce synchronous blocking. For safer authorization steps, a Redis-based smart cache (with TTL management) is used; when the trip ends, the TripEnded event is produced via Kafka to decouple the payment process from the main flow. While the Payment Service consumes this event and performs collection in the background, a settlement decision logic is applied for differences between the estimated amount and the final amount. For failure scenarios, staged fallback, debt creation, and scheduled automatic collection attempts are used to maintain collection continuity; additionally, for certain risky cases, the flow is designed to redirect to stronger verification steps such as 3D Secure.

Within the scope of the paper, the impact of the approach is evaluated using both technical performance (waiting/blocking times) and field operational metrics. Production data shows that the high client-side waiting times in the synchronous flow decrease noticeably with the asynchronous "trigger and continue" approach, and that the vehicle exit time at trip end improves measurably. In this way, the payment step is transformed from a "stop" that breaks the natural flow of the trip into a fast background process that is as invisible as possible. Following this introduction, the paper systematically presents architectural components, data flow, fault-tolerance design, and KPI results.

II. Literature Review

The payment experience in mobility applications is a critical touchpoint that shapes user satisfaction. A significant part of the literature on digital payment and technology acceptance shows that as users' perceived "convenience" increases, both intention to use and continued use tendency become stronger. For example, the Technology Acceptance Model (TAM) emphasizes perceived usefulness and perceived ease of use as key determinants of adoption [1]. The UTAUT approach expands this with factors such as performance expectancy, social influence, and facilitating conditions, especially for mobile and service-based technologies [2]. Comprehensive reviews in mobile payments report that users often decide based on speed/convenience, perceived security, and system reliability, and that friction in the payment step (extra clicks, waiting, failed attempts) can increase abandonment rates [3]. In ride-hailing contexts, similarly, perceived service quality and transaction convenience affect user attitude and intention, and closure steps such as payment can strongly shape the experience [4].

Making the payment experience "invisible" (collection completed in the background without the user giving final approval) is often associated with the goal of "frictionless/non-blocking collection." However, this goal does not mean removing security steps entirely; rather, it requires a staged, risk-based design that requests additional verification when needed. In card payments, 3-D Secure (3DS) standards were developed to provide this balance, supporting two main flows: a "frictionless" flow (authentication without extra user steps) for low-risk transactions and a "challenge" flow (additional verification) for risky transactions [5]. Therefore, in mobility scenarios where collection is desired without manual user approval, the literature highlights that while friction is reduced, security must be managed dynamically using risk score, dispute/chargeback likelihood, and fraud indicators [5].

The literature also notes that in asynchronous collection scenarios such as "pre-authorized payment," architectural design is at least as decisive as UX. In microservice-based systems, payment is the combined output of multiple components such as trip, pricing, campaign, and billing, making classic distributed transaction approaches challenging in practice. One of the best-known solution lines is the "Saga" model: a long-running workflow is split into local transactions in each service, and consistency is maintained via compensating actions in case of failure [6]. Recent work also shows that the saga approach has become a common pattern in event-driven microservices and can be applied via different frameworks (e.g., orchestration/choreography) [10]. This provides a strong foundation for processes like payment that must have a "single effective outcome," in terms of scalability and fault tolerance [6], [10].

Event-driven design naturally brings up "eventual consistency." In large-scale systems, high availability may not always be achieved together with strong consistency at every moment; therefore, many practical systems accept delayed consistency and manage it with good design principles [7]. More advanced studies discuss limitations of eventual consistency and how it can be measured and controlled at the application layer (e.g., duration and impact of inconsistency) to support more informed trade-offs [8]. For reliable transport of event streams, high-throughput and low-latency messaging infrastructures (e.g., Kafka) are well-known building blocks [9]. Finally, for the reliability of payment flows, "idempotency" (producing the same outcome when the same request is repeated) is considered critical against repeated requests, network disconnects, and retries; idempotency keys and design patterns are recommended to reduce duplicate-charge risk, especially for side-effect operations such as collection [11]. Together, this background positions the mobility-specific approach of "close the trip, complete collection safely in the background" on a literature-aligned basis for both UX and system resilience.

III. Methods

The method followed in this study is based on designing an architecture that separates the trip "finish" step from the synchronous flow that waits for payment approval, and

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

completes the payment process asynchronously in the background. The literature has long emphasized that it becomes difficult to guarantee strong consistency and high availability at the same time in distributed systems, and that event-driven designs based on eventual consistency are therefore often preferred [7]. It is also discussed extensively that such approaches can provide independent scalability and fault isolation in microservices [12], [16]. In this project, because payment is a domain that requires consistency, an Event-Driven Microservices Architecture was adopted to reduce synchronous blocking; asynchronous messaging and caching mechanisms were positioned as the backbone of the solution.

The applied technical methodology was executed in five main phases. In the first phase, to safely start the "pre-authorized" flow, a mandate/recurring token was created on the user's card, and mandate checks were performed on triggers such as card listing, adding a card, and starting a ride. In the second phase, a Redis-based smart cache was developed to reduce unnecessary authorization attempts and the load on the bank side; successful authorization information was stored with a 2-hour TTL so that no new authorization was taken from the same card within this period. In the third phase, when the trip ends, the Trip Service writes the "TripEnded" event to a Kafka topic to decouple payment from the main flow; the Payment Service consumes this event and starts background collection. This matches the distributed messaging approach recommended in the literature for carrying high-volume event streams with low latency [9].

In the fourth phase, a settlement algorithm was applied for differences between the estimated amount and the taxi meter amount: if the authorized amount was larger than the actual amount, the authorization was cancelled and the actual amount was charged via "direct recurring"; if the authorized amount was sufficient, it was finalized with "recurring post-auth." In the fifth phase, exception handling was designed for risky transactions; in high-amount or fraud-flagged cases, the pre-authorized flow was interrupted and the user was redirected to 3D Secure verification. This aligns with the risk-based "frictionless/challenge" logic in 3DS standards [5]. Finally, for failure scenarios, a staged fallback was implemented: if the primary card fails, other mandated cards are tried in sequence; if all attempts fail, a "pending payment/debt" record is created and periodic automatic collection is attempted via cron jobs.

In such distributed workflows, the saga approach (local steps + compensation logic) is positioned as one of the fundamental solution patterns in the literature [6], [10]; and idempotent design principles are considered critical to reduce duplicate-transaction risk under retry conditions [14], [15]. The development process followed agile principles, including two-week sprints, traceable task tracking via Jira, code reviews via GitHub pull requests, unit tests, and Jenkins-based CI/CD automations to assure quality. For validation, measurements were taken using real production data; client-side blocking times of the old synchronous structure were compared with those of the new asynchronous structure. Additionally, operational metrics such as passenger drop-off time were monitored together with driver feedback to quantitatively evaluate the impact on both technical performance and real operational flow.

IV. RESULTS

The effects of the pre-authorized and asynchronous payment architecture developed in this study were measured using real production data, both in terms of technical performance and field operations. The primary goal was to quantify the "waiting and uncertainty" problem at the end of the trip in the old synchronous structure and show to what extent the new architecture reduces this waiting. This aligns with the literature emphasizing that trade-offs among consistency, availability, and latency are inevitable in distributed systems, and that some steps can be moved to the background and managed with eventual consistency to protect user experience [7].

The most prominent gain in technical performance is the dramatic reduction of client-side "blocking" time. While the screen was locked until payment confirmation arrived in the old synchronous structure, in the new system the payment process is moved to the background, so the client-side waiting is reduced largely to network-latency scale. Measurements show that the average client blocking time decreased from 46.33 seconds to approximately 2.31 seconds, corresponding to about a 95.01% improvement; P95 decreased from 62.67 seconds to approximately 9.25 seconds, and P99 decreased from 68.70 seconds to approximately 9.58 seconds.

This result shows that decoupling the end-of-trip flow with a "fire-and-forget" logic practically removes user waiting. In this decoupling, using messaging infrastructures such as Kafka as a suitable base for transporting event streams with low latency is a known approach [9]. On the operational side, the metric most directly reflected in user experience was the vehicle exit (drop-off) time. Based on field tests and driver feedback, the old process (payment confirmation/POS connection/manual steps) added an average closure delay of 4.67 seconds (SD $\approx \pm 1.4$ s), while in the new system this decreased to 1.43 seconds (SD $\approx \pm 0.5$ s).

This corresponds to up to a 3.2× acceleration in passenger exit speed, indicating that "micro delays" at closure can have meaningful practical impact—especially in locations where stopping is difficult and circulation is critical (e.g., airports and shopping malls). Two financial findings stand out: faster collection and recovery of unjust cancellations. It was reported that currently about 25% of card-based trips are processed via the pre-authorized payment infrastructure. Despite this partial share, the "collection rate within the first hour" increased from 38.56% in April–May to 47.23% in June, and to 79.88% in July.

In addition, in July it was reported that 77% of unjust cancellations originating from Masterpass within a total volume of 31k were successfully collected thanks to the recurring payment infrastructure. It was also noted that background jobs for failed transactions, retry logic, and the efficiency of collection automation were decisive in achieving these gains. In the literature, saga-style local steps + compensation logic for long-running distributed workflows [6] and the importance of idempotent design under retry conditions [14] are core frameworks that explain the reliability of such collection processes.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

Finally, the impact of these technical and financial improvements on operations teams was also measured. With automatic collection in place, critical driver complaints such as "the customer got out without paying" and "the trip ended but my money didn't arrive" were minimized; in parallel, a 20.24% decrease in call volume was reported. Overall, the findings show that the pre-authorized and asynchronous approach (i) significantly reduces user waiting at trip closure and improves smoothness, (ii) shortens drop-off time, and (iii) increases collection speed and recovery while reducing operational workload.

V. DISCUSSION

The findings show that decoupling the payment step from trip closure and completing it in the background strengthens the expectation of a "smooth closure" in mobility scenarios. The reduction in client-side waiting times and the shorter vehicle exit time should be seen not only as a performance improvement, but also as a design choice that steers the availability–consistency trade-off in distributed systems in favor of user experience [1]. As described, the key design decision is to produce the TripEnded event, detach payment from the main flow, and avoid blocking the mobile client with a "trigger and continue" approach. The literature emphasizes that forcing strong consistency at every step can increase latency, while eventual consistency can be a practical strategy at scale by allowing short-term consistency delay and returning a fast response to the user [1]. In this project, while payment confirmation and settlement proceed in the background, the passenger app can move to the "payment summary" screen and the closure moment becomes faster.

The second discussion axis is that this speed gain changes the "location of risk." As explicitly noted, when payment confirmation happens after the passenger leaves the vehicle, the collection risk shifts to the company side. This matches the saga framing, where long-running workflows in microservice architectures are managed with local steps and compensating actions [2]. Accordingly, progressive fallback, debt creation, and periodic automatic collection (cron jobs) are designed as an operational strategy aligned with this compensation/improvement logic. Because retries and ordering uncertainties in distributed systems can create duplicate-transaction risk, idempotent design principles are critical—especially for side-effect operations such as payment [3]. Therefore, the sustainability of the "asynchronous speed" gain is directly tied to safe retry design, uniqueness of transaction identities, and observability practices (monitoring/alerting).

The third major topic is security and abuse risk. The report notes that reducing approval steps can open the door to malicious use, and that some drivers were detected trying to create "non-existent trips" using fake accounts. To manage this risk, fraud detection algorithms, device and GSM-based limits, and constraints based on transaction frequency and matching patterns were introduced. This aligns with the idea of risk-based staged authentication in payment security (frictionless when risk is low, additional verification when risk increases); 3D Secure's frictionless/challenge flows are industry-standard counterparts of this approach [4]. In the project, stopping the flow and redirecting to 3D Secure for transactions above certain amounts or marked as suspicious is the concrete application of this "dynamic security" design.

In conclusion, the study shows that the asynchronous pre-authorized payment approach (i) improves passenger and driver experience by reducing friction at the end of the trip, (ii) increases collection performance while lowering operational workload, and (iii) in return makes the need for risk management, observability, and fraud controls more critical. As a future vision, moving a large portion of the remaining transaction volume to the pre-authorized and asynchronous infrastructure and increasing the share of card payments are targeted. When evaluated together with microservices' independent evolution and scalability advantages [5], [6], the system appears open to further progress in both productization and technical maturity. In addition, studies proposing boundaries/extensions for operating eventual-consistency-based systems safely and predictably [7] provide a methodological background for measuring and managing consistency delay in such payment flows. The overall takeaway is that the "non-blocking collection" goal is not only a UX improvement; it is a holistic transformation that requires architecture and risk management disciplines to be designed together.

ACKNOWLEDGMENT

The current study was performed in the Research & Development Center of an urban mobility / ride-hailing company that was funded by the Ministry of Industry and Technology.

REFERENCES

[1] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, Sep. 1989, doi: 10.2307/249008.

[2] V. Venkatesh, M. G. Morris, G. B. Davis, and F. D. Davis, "User acceptance of information technology: Toward a unified view," *MIS Quarterly*, vol. 27, no. 3, pp. 425–478, Sep. 2003, doi: 10.2307/30036540.

[3] T. Dahlberg, J. Guo, and J. Ondrus, "A critical review of mobile payment research," *Electronic Commerce Research and Applications*, vol. 14, no. 5, pp. 265–284, 2015, doi: 10.1016/j.elerap.2015.07.006.

[4] S. A. H. Shah and H. Kubota, "Analyzing travelers' attitude towards ride-hailing services in developing countries: Case of Lahore, Pakistan," *IATSS Research*, vol. 46, pp. 223–235, 2022.

[5] EMVCo, "3-D Secure Specification v2.2.0," EMVCo, documentation page, accessed: 15 December 2025.

[6] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. on Management of Data (SIGMOD '87)*, 1987, pp. 249–259, doi: 10.1145/38713.38742.

[7] W. Vogels, "Eventually consistent," *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, Jan. 2009, doi: 10.1145/1435417.1435432.

[8] P. Bailis and A. Ghodsi, "Eventual consistency today: limitations, extensions, and beyond," *Communications of the ACM*, vol. 56, no. 5, pp. 55–63, May 2013, doi: 10.1145/2447976.2447992.

[9] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB*, 2011, pp. 1–7.

[10] K. M. Koyya and B. Muthukumar, "A Survey of Saga Frameworks for Distributed Transactions in Event-driven Microservices," in *2022 Third International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)*, 2022, pp. 1–6, doi: 10.1109/ICSTCEE56972.2022.10099533.

[11] A. Bhole, "Ensuring Idempotency in Java RESTful APIs," *International Journal of Innovative Research and Creative Technology (IJIRCT)*, vol. 8, no. 5, 2022.

[12] N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," arXiv preprint arXiv:1606.04036v4, 2017.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

[13] E. Daraghmi, A. Darwish, and M. Yuan, "Enhancing Saga Pattern for Distributed Transactions within Microservices Architecture," *Applied Sciences*, vol. 12, no. 12, 2022.
[14] P. Helland, "Idempotence Is Not a Medical Condition," *Communications of the ACM*, vol. 55, no. 5, 2012, doi: 10.1145/2160718.2160734.
[15] R. Fielding et al., "HTTP Semantics," RFC 9110, 2022.
[16] P. Di Francesco, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," *Journal of Systems and Software*, vol. 150, pp. 77–97, 2019, doi: 10.1016/j.jss.2019.01.001.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.